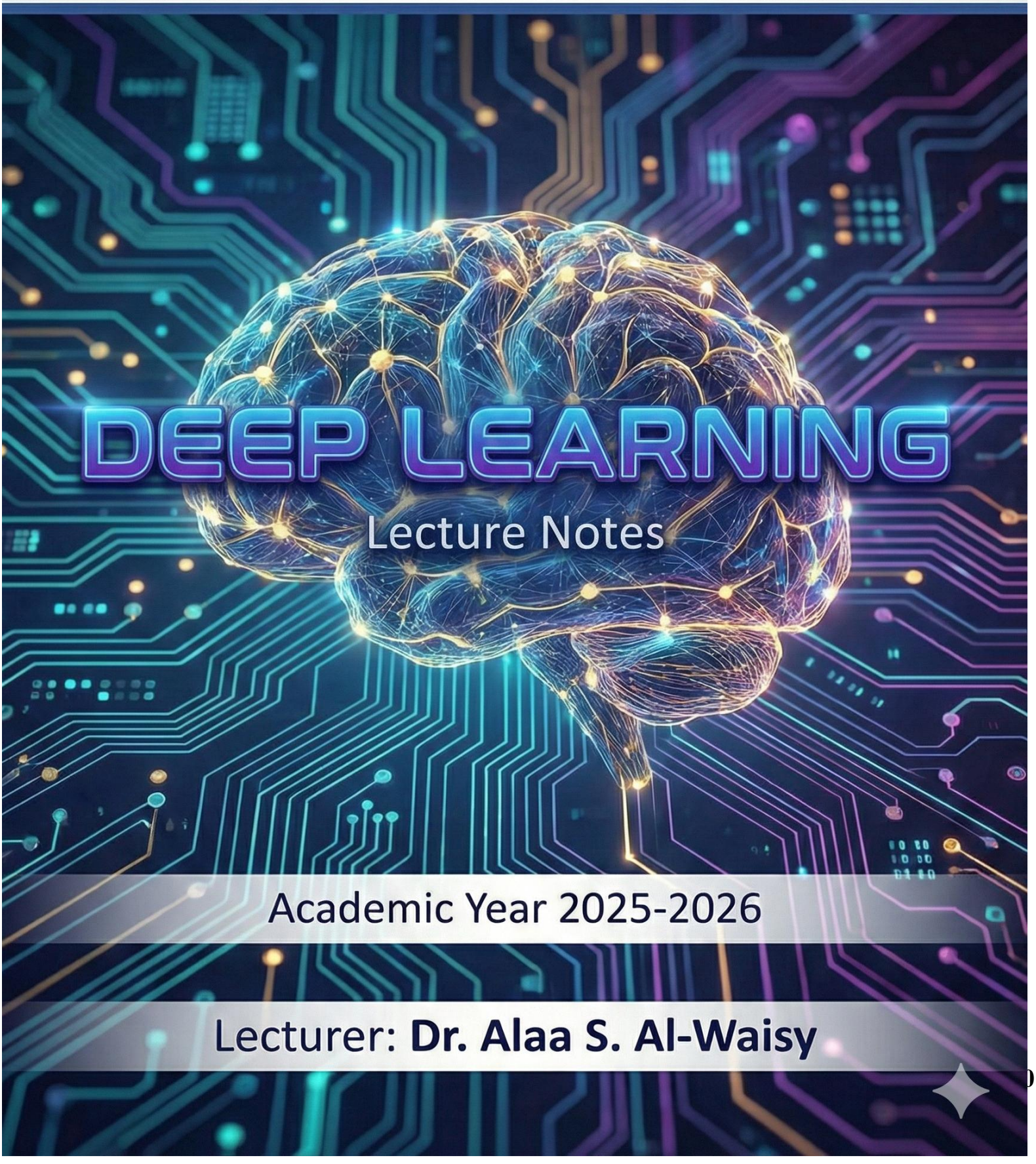

DEEP LEARNING
Lecture Notes
Academic Year 2025-2026
Lecturer: Dr. Alaa S. Al-Waisy

Lecture Three

Introduction to Convolutional Neural Networks (CNNs)

1. What is a Convolutional Neural Network?

- **A Convolutional Neural Network (CNN or ConvNet)** is a specialized type of deep learning model designed primarily for processing and analyzing data with a grid-like structure, such as images and videos (See Figure 1). Inspired by the human visual cortex, CNNs are exceptionally effective at identifying patterns in visual data, making them the cornerstone of modern computer vision.
- CNNs are very similar to ordinary Neural Networks from the previous lecture: they are made up of neurons that have **learnable weights** and **biases**. Each neuron receives some **inputs**, performs a **dot product**, and optionally follows it with a **non-linearity** (using **activation functions**). The entire network still expresses a single, differentiable score function, ranging from raw image pixels at one end to class scores at the other. And they still have a loss function (e.g., **cross-entropy**) on the last (fully-connected) layer, and all the tips/tricks we developed for learning regular Neural Networks still apply.
- Thus, what changes? To understand a CNN, it's helpful to first understand the problem it solves. The problem with traditional Neural Networks. If you wanted to use a standard Neural Network (like a **Multi-Layer Perceptron**) to classify an image, you would face two major problems:
  1. **Loss of Spatial Information:** A standard network requires the input to be a one-dimensional vector. To feed it an image, you would have to "**flatten**" the grid of pixels into one long row. This process destroys the spatial relationships between pixels. The network loses the crucial information that certain pixels are above, below, or next to others, making it very difficult to detect shapes and objects.
  2. **Parameter Explosion:** A small ($\mathbf{224 \times 224}$) pixel color image has ($\mathbf{224 \times 224 \times 3}$) $= \mathbf{150,528}$ input values. If the first hidden layer of your network had just **1,000** neurons, you would need over **150 million** weights for that single layer alone. This is computationally massive, slow to train, and makes the network extremely prone to overfitting.
- CNNs were cleverly designed to solve these two problems.

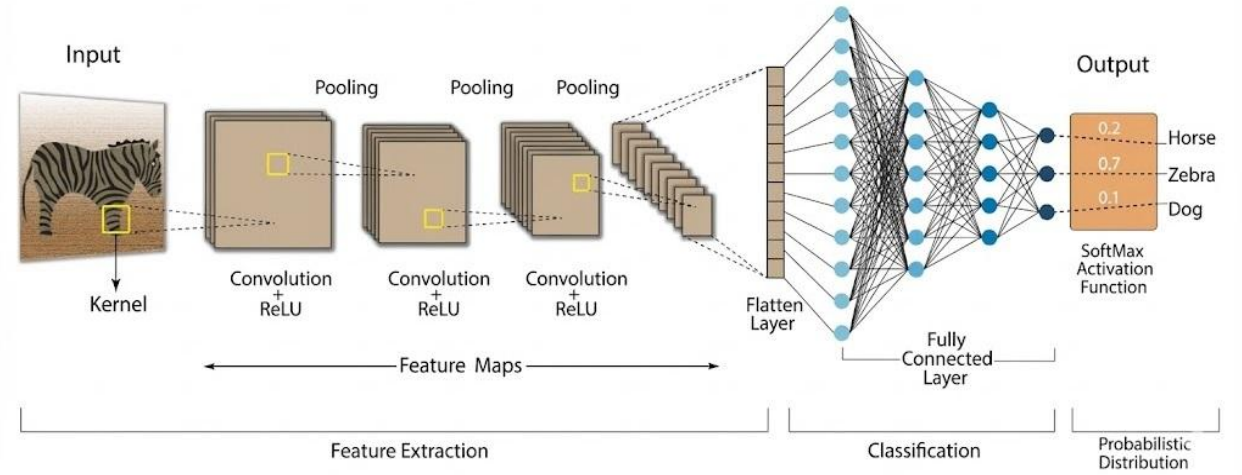


Figure 1: An example of CNN architecture.

2. Core Principles of CNNs

- Instead of looking at every pixel at once, a CNN works by looking at small, localized patches of an image to find simple patterns. It then combines these simple patterns in subsequent layers to form more complex patterns (See Figure 2).

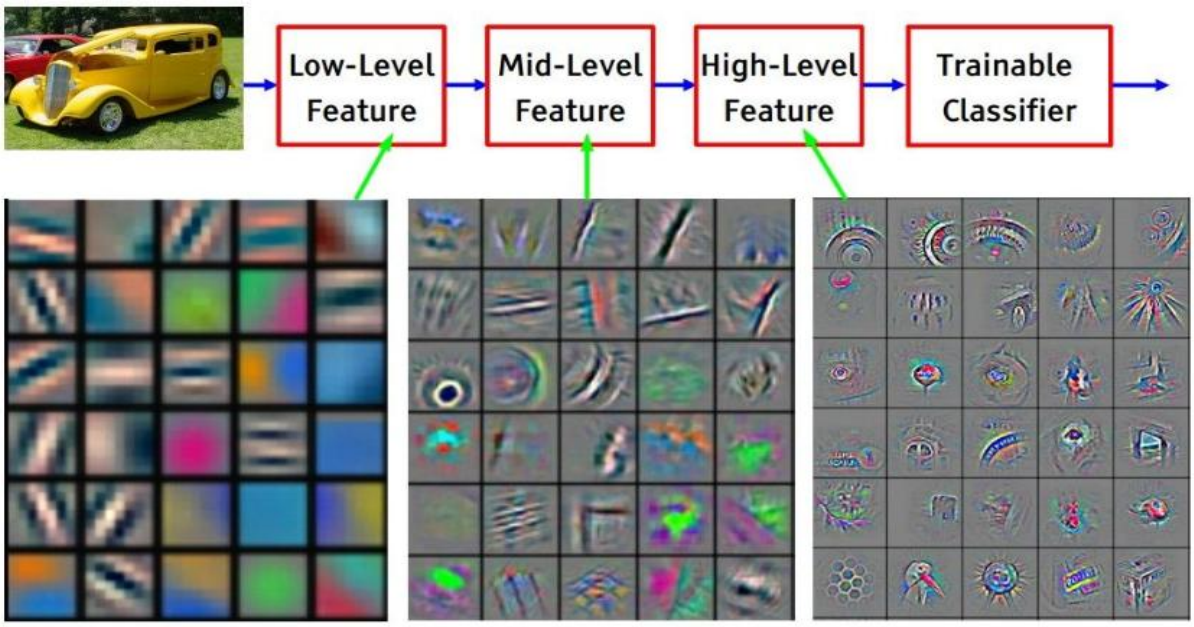


Figure 2: Features visualization of CNN trained on ImageNet

1. Local Connectivity: When dealing with high-dimensional inputs such as images, as we saw above it is impractical to connect neurons to all neurons in the previous volume. Instead, we will connect each neuron to only a local region of the input volume. The spatial extent of this connectivity is a hyperparameter called

the **receptive field** of the neuron (equivalently this is the **filter size**). The extent of the connectivity along the depth axis is always equal to the depth of the input volume (See Figure 3). It is important to emphasize again this asymmetry in how we treat the spatial dimensions (**width** and **height**) and the depth dimension: The connections are local in 2D space (along **width** and **height**), but **always full along the entire depth of the input volume.**

a. Instead of connecting every input pixel to all neurons (as in fully connected networks), CNNs use **small filters (kernels)** that scan *local regions* of the input (e.g., a **3×3** patch of pixels).

b. *Why it matters*: Reduces parameters dramatically. For a **(100×100×3)** image:

i. A fully connected-layer needs **30,000×1,000=30M** weights.

ii. A 3×3 convolutional layer needs only **3×3×3×64=1,728** weights (for **64 filters**).

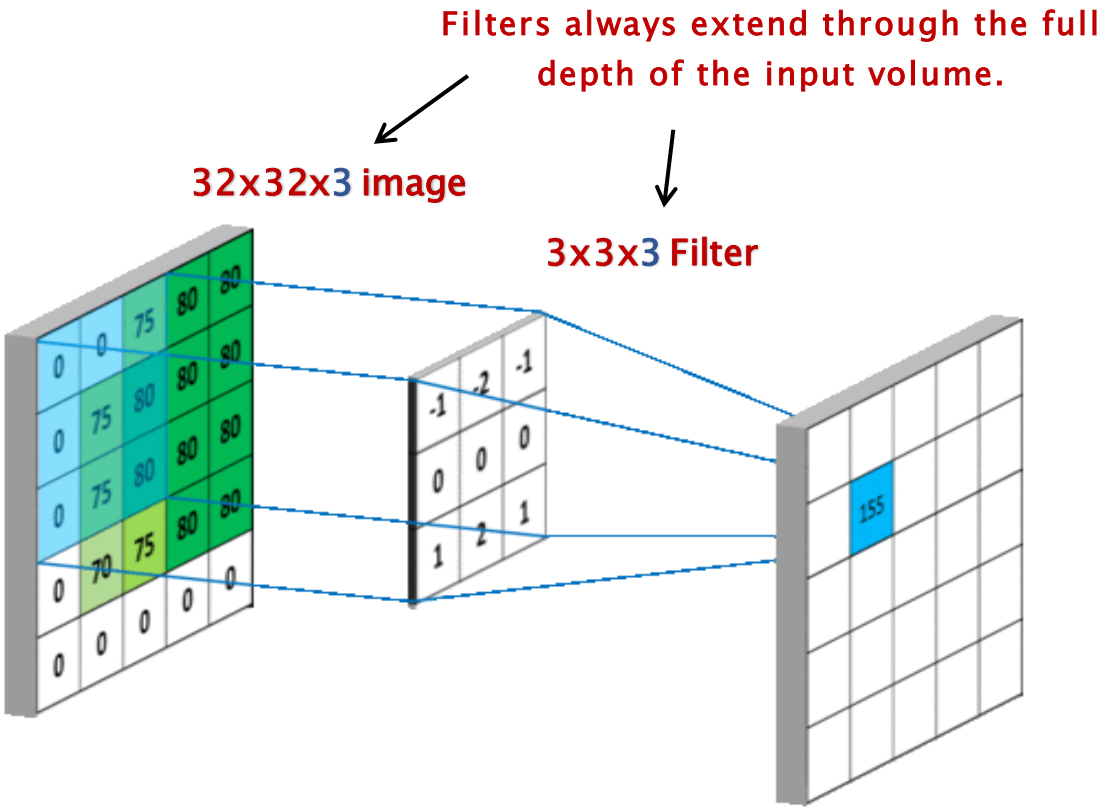


Figure 3: An example of Convolution operation (an element-wise multiplication) in CNN.

Example 1: For example, suppose that the input volume has size [**32x32x3**], (e.g. an RGB CIFAR-10 image). If the receptive field (or the filter size) is (**5x5**), then each neuron in the ***Conv Layer*** will have weights to a [**5x5x3**] region in the input volume, for a total of **5*5*3 = 75** weights (and **+1** bias parameter). These weights for one filter. If you have **10 filters** in the ***Conv Layer****, then the overall weights in this layer is* **(76 × 10=760 parameters)**. Notice that the extent of the connectivity along the depth axis must be **3**, since this is the depth of the input volume.

Example 2: Suppose an input volume had size **[16x16x20]**. Then using an example receptive field size of **3x3**, every neuron in the ***Conv Layer*** would now have a total of **3*3*20 = 180** weights (and **+1** bias parameter) connect to the input volume. These weights for one filter. If you have **5 filters** in the ***Conv Layer****, then the overall weights in this layer is* **(181 × 5=905 parameters)**. Notice that, again, the connectivity is local in 2D space (e.g., **3x3**), but full along the input depth (**20**).

2. Parameter sharing scheme Parameter sharing in a CNN means the same filter weights are used across all spatial locations of an input feature map (See Figure 4). A single kernel slides over the image and applies identical weights at every position. This reduces the number of trainable parameters compared to fully connected layers. It also enforces translation consistency because the network detects the same pattern anywhere in the image. For example, one 3×3 filter learns an edge pattern and responds to that edge whether it appears at the top or bottom of the image. This design lowers memory use, speeds training, and improves generalization by limiting overfitting.

a. The same filter slides across the entire input (via **convolution**), detecting features (e.g., **edges, textures**) *regardless of position*.

b. **Output*:* A feature map** highlighting where the filter's pattern occurs.

c. **Example 1*:*** A vertical-edge detector activates at all locations with vertical lines.

d. **Example 2:** Consider a (**3 × 3**) filter sliding over a (**5 × 5**) grayscale image. Without parameter sharing. Then, each position of the filter would have its own set of **9** weights.

Total parameters: 3 × 3 × 9 = 81

With parameter sharing scheme:

The filter has only **9** weights.

Total parameters: 9

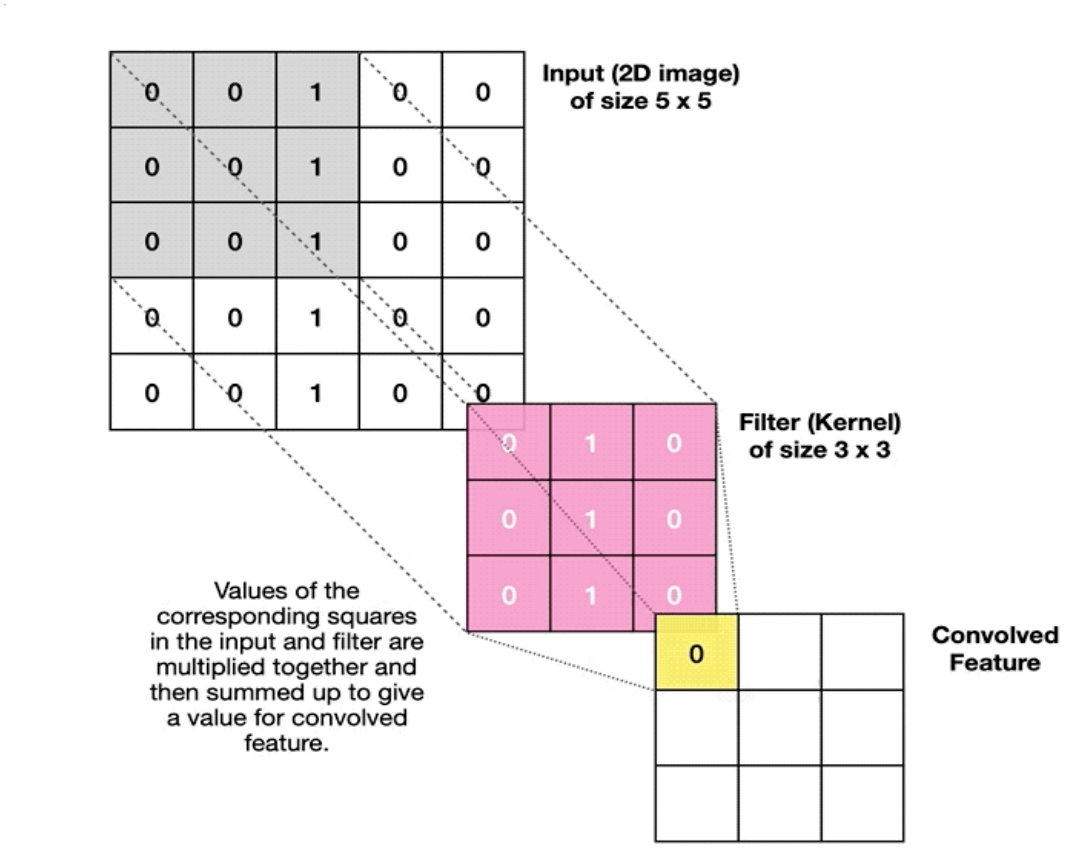


Figure 4: An example of the parameter sharing scheme in the convolution layer.

3. **Spatial Hierarchy**
   a. **Early layers** detect simple features (edges, colors).
   b. **Deeper layers** combine these into complex patterns (e.g., eyes → faces → objects).
   c. ***Biological inspiration*:** Mimics the human visual cortex's hierarchical processing.

Example: Think of it like building a face:

- **Layer 1:** The network learns to detect very simple features like **edges**, **corners**, and **color gradients**.
- **Layer 2:** It combines the **edges** and **curves** from the first layer to learn to recognize more complex features like **eyes**, **noses**, and **mouths**.
- **Layer 3:** It combines the eyes, nose, and mouth to recognize a complete **face**.

This ability to learn a **hierarchy of features** is the fundamental power of a CNN (See Figure 1).

3. The Main Building Blocks of a CNN

As we described above, a simple CNN is a sequence of layers, and every layer of a CNN transforms one volume of activations to another through a differentiable function. We use three main types of layers to build CNN architectures: **Convolutional Layer**, **Pooling Layer**, and **Fully-Connected Layer** (exactly as seen in regular Neural Networks). We will stack these layers to form a full CNN **architecture**.

1. The Convolutional Layer

This is the core building block of the CNN. Its job is to detect features. It does this using **filters** (also called **kernels**).

- **Filter:** A small matrix of weights (e.g., **3x3** or **5x5**). Each filter is specialized to detect one type of feature, like a vertical **edge**, a specific **color**, or a particular **texture**. A key point is that the network *learns* the values of these filters during training.
- **Convolution Operation:** The filter slides over the input image, patch by patch. At each position, it performs **an element-wise multiplication** with the part of the image it's currently on and sums the results into a single output pixel (See Figure 3-4). Mathematically, for a 2D input **I** and a kernel **K**, the value at a specific position in the output feature map S is calculated as:

$$\boldsymbol{S(i,j) = (I * K)(i,j) = \sum_{m} \sum_{n} I(i+m, j+n) * K(m,n)}$$

  Where:
  - $\boldsymbol{i, j}$ are the positions in the output.
  - $\boldsymbol{m, n}$ iterate over the kernel dimensions.
  - After this operation, a non-linear activation function (usually **ReLU**) is applied to introduce non-linearity, allowing the network to learn complex patterns.
- **Feature Map (or Activation Map):** The output of this sliding operation is a new 2D grid called *a feature map*. This map highlights the locations in the original image where the filter's specific feature was detected.
- As shown in Figure 5, a single convolutional layer will have **many different filters**, each producing its own feature map, thereby detecting many different features in parallel.

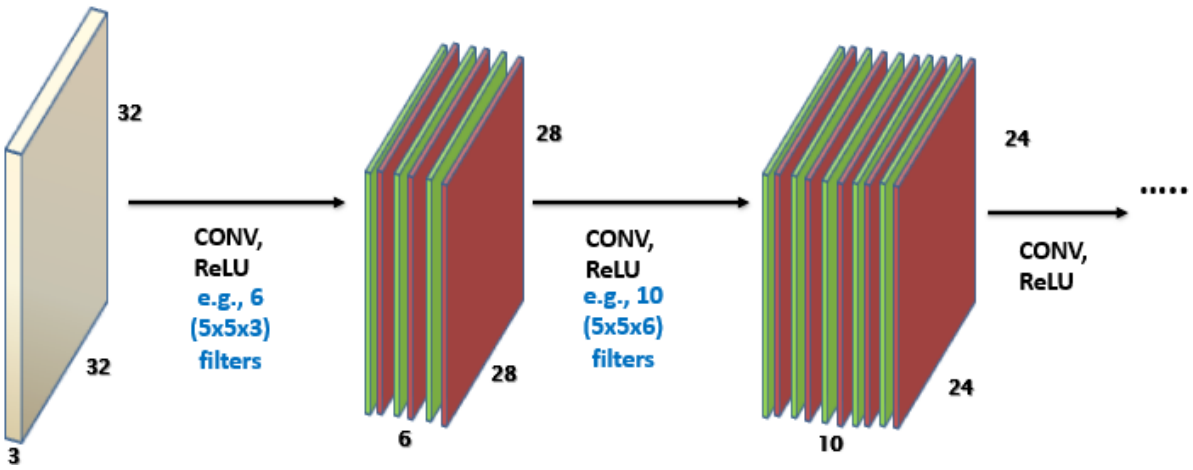


Figure 5: Preview of an CNN is a sequence of Convolutional layers, interpreted with activations maps.

- **Mathematical Example:**

Input Data

Let's assume we have:

- A **5×5 grayscale image** (1 channel)

1	**2**	**3**	**4**	**5**
6	**7**	**8**	**9**	**10**
11	**12**	**13**	**14**	**15**
16	**17**	**18**	**19**	**20**
21	**22**	**23**	**24**	**25**

- A **3×3 convolutional filter/kernel**

1	**0**	**-1**
1	**0**	**-1**
1	**0**	**-1**

- **Stride = 1** (moving the filter 1 pixel at a time)
- **Padding = 0** (no padding)
- **Bias: b = 1**

Convolution Operation

Output size calculation:

- **Output height** = **(Input height - Filter height + 2×Padding)/Stride + 1** = (5-3+0)/1 + 1 = 3
- **Output width** = **(Input width - Filter width + 2×Padding)/Stride + 1** = (5-3+0)/1 + 1 = 3

So, the output will be a **3×3 feature map**.

Step-by-step computation:

1. **Top-left position** (filter aligned with rows **0-2**, columns **0-2**):

Input region:	**Filter:**
[1, 2, 3]	[1, 0, -1]
[6, 7, 8]	[1, 0, -1]
[11,12,13]	[1, 0, -1]

Element-wise multiplication:

1×1 + 2×0 + 3×(-1) + 6×1 + 7×0 + 8×(-1) + 11×1 + 12×0 + 13×(-1)

= 1 + 0 - 3 + 6 + 0 - 8 + 11 + 0 - 13 = -6

Add bias: -6 + 1 = -5

Top-middle position (filter aligned with rows **0-2**, columns **1-3**):

Input region:	**Filter:**
[2, 3, 4]	[1, 0, -1]
[7, 8, 9]	[1, 0, -1]
[12,13,14]	[1, 0, -1]

Computation:

2×1 + 3×0 + 4×(-1) + 7×1 + 8×0 + 9×(-1) + 12×1 + 13×0 + 14×(-1) + 1

= 2 - 4 + 7 - 9 + 12 - 14 = -6

Add bias: -6 + 1 = -5

Continue for all positions...

Final 3×3 output feature map:

O = [[-5, -5, -5],
[-5, -5, -5],
[-5, -5, -5]]

```
import numpy as np

def manual_convolution_2d(input_img, kernel, bias, stride=1,
padding=0):
    # Manual 2D convolution implementation
    # Add padding if needed
    if padding > 0:
        input_padded = np.pad(input_img, padding,
mode='constant')
    else:
        input_padded = input_img
    # Get dimensions
    input_h, input_w = input_padded.shape
```

```
    kernel_h, kernel_w = kernel.shape

    # Calculate output dimensions
    output_h = (input_h - kernel_h) // stride + 1
    output_w = (input_w - kernel_w) // stride + 1

    # Initialize output feature map
    output = np.zeros((output_h, output_w))

    # Perform convolution
    for i in range(0, output_h):
        for j in range(0, output_w):
            # Extract the region from input
            region = input_padded[i * stride:i * stride +
kernel_h,
            j * stride:j * stride + kernel_w]

            # Element-wise multiplication and sum
            conv_sum = np.sum(region * kernel)

            # Add bias
            output[i, j] = conv_sum + bias

    return output

# Example usage with our data
input_image = np.array([[1, 2, 3, 4, 5],
                        [6, 7, 8, 9, 10],
                        [11, 12, 13, 14, 15],
                        [16, 17, 18, 19, 20],
                        [21, 22, 23, 24, 25]])
kernel = np.array([[1, 0, -1],
                   [1, 0, -1],
                   [1, 0, -1]])
bias = 1
# Apply convolution
result = manual_convolution_2d(input_image, kernel, bias)
print("Manual Convolution Result:")
print(result)
print(f"\nShape: {result.shape}")
```

- **Key Hyperparameters of Convolutional Layer**

  The behavior of the convolutional layer is controlled by specific hyperparameters that you must define when creating the layer.

  A. Kernel Size (Filter Size)

  The dimensions of the sliding window.

  - **Notation:** Usually odd numbers: **3×3**, **5×5**, **7×7**.

- **Impact:** A larger kernel sees a bigger portion of the image at once (more global context) but is computationally more expensive and loses fine-grained detail. A **3×3** kernel is most common as it captures local textures efficiently.

B. Stride

The number of pixels the kernel moves at each step.

- **Stride 1:** The kernel moves one pixel at a time. This results in a high-resolution output map but is computationally heavy.
- **Stride 2:** The kernel jumps two pixels. This down-samples the input, reducing the spatial dimensions (width/height) of the output significantly.
- **Impact:** As shown in Figure 6, a higher stride leads to smaller output dimensions. **Stride = 1** and **Stride = 2** would result to a **5 x 5** and **3 x 3** feature map respectively:
- **What is the size of the output when applying a step of 3?**

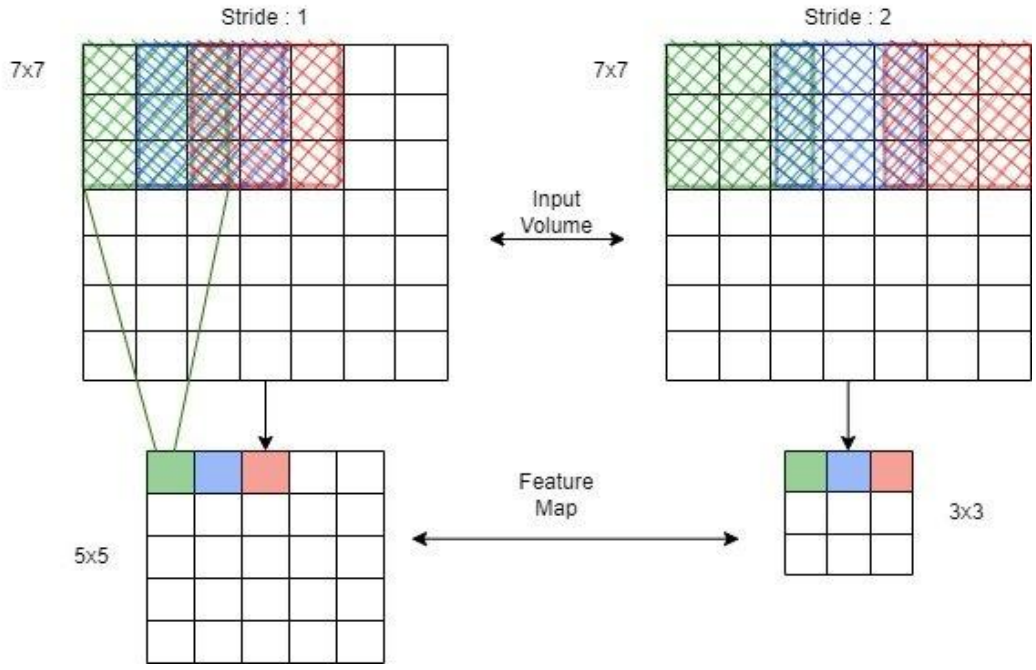


Figure 6: The impact of the stride value on the output of the Convolutional layer.

C. Padding

Since the convolution reduces the size of the image (pixels on the edges are visited less often or not at all), we often add borders of zeros around the input, as shown in Figure 7.

- **Valid (No Padding):** The kernel stays strictly inside the image. Output size decreases.

- **Same (Zero Padding):** Padding is added so the output spatial dimensions equal the input dimensions (if stride is **1**).
- **Impact:** Padding helps preserve information at the borders and allows you to build deeper networks without the feature maps shrinking to (**0x0**) too quickly.

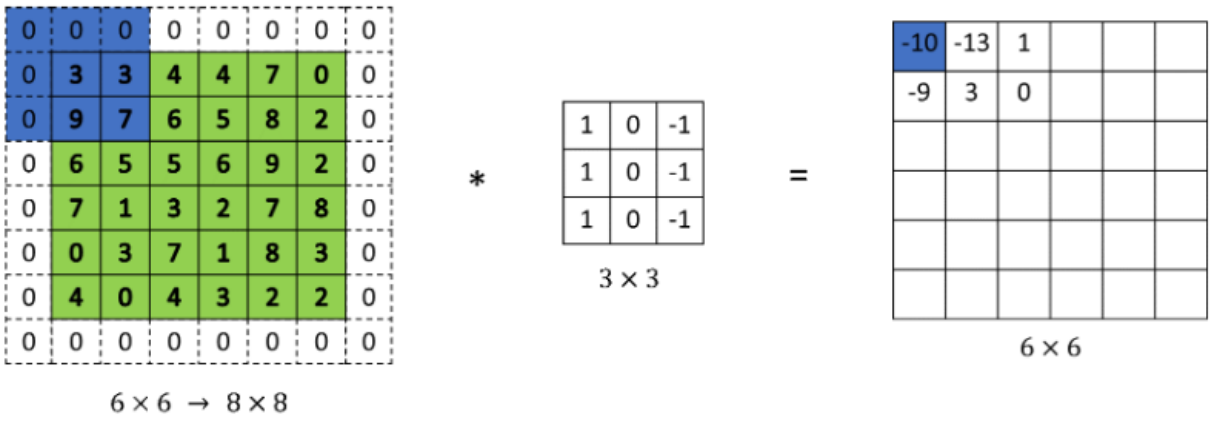


Figure 7: Applying padding of 1 before convolving with (3×3) filter.

- In general, common to see Convolutional layer with **stride 1**, filter size (**F × F**), and zero-padding with **(F-1)/2**. This will preserve the size of input volume.

  e.g., F = 3 => zero padding with 1

  F = 5 => zero padding with 2

  F = 7 => zero padding with 3

D. Number of Filters (Depth)

How many different kernels the layer will learn.

- If you have **32** filters, the layer will output **32** distinct feature maps (one for each filter).
- One filter might learn to detect vertical edges, another horizontal edges, another circles, etc.
- **Impact:** More filters allow the model to learn more complex features but increase the number of parameters (memory and computation).

➢ **Summary:** To summarize, the Conv Layer:

- Accepts a volume of size $\mathbf{W_1} \times \mathbf{H_1} \times \mathbf{D_1}$
- Requires four hyperparameters:
  - Number of filters **K**,
  - Their spatial extent **F**,
  - The stride **S**,

- The amount of zero-padding **P**.

- Produces a volume of size $\mathbf{W_2} \times \mathbf{H_2} \times \mathbf{D_2}$ where:
  - $\mathbf{W_2} = (\mathbf{W_1} - \mathbf{F} + \mathbf{2P})/\mathbf{S} + \mathbf{1}$
  - $\mathbf{H_2} = (\mathbf{H_1} - \mathbf{F} + \mathbf{2P})/\mathbf{S} + \mathbf{1}$ (i.e. width and height are computed equally by symmetry).
  - $\mathbf{D_2} = \mathbf{K}$
- With parameter sharing, it introduces $\mathbf{F} * \mathbf{F} * \mathbf{D_1}$ weights per filter, for a total of $(\mathbf{F} * \mathbf{F} * \mathbf{D_1}) * \mathbf{K}$ weights and $\mathbf{K}$ biases.
- In the output volume, the $\boldsymbol{d} - \boldsymbol{th}$ depth slice (of size $\mathbf{W_2} \times \mathbf{H_2}$) is the result of performing a valid convolution of the $\boldsymbol{d} - \boldsymbol{th}$ filter over the input volume with a stride of **S**, and then offset by $\boldsymbol{d} - \boldsymbol{th}$ bias.
- **Common sittings:**

  K = (powers of 2, e.g., 8, 16, 32, 64, 128, 256, 512)

  F = 3, S = 1, P = 1

  F = 5, S = 1, P = 2

  F = 5, S = 2, P =? (whatever fit)

  F = 1, S = 1, P = 0

Example: The input volume is of size $W_1 = 5$, $H_1 = 5$, $D_1 = 3$, and the CONV layer parameters are **K = 2, F = 3, S = 2, P = 1**. That is, we have **two** filters of size (**3×3**), and they are applied with a stride of **2**. Therefore, the output volume size has spatial size **(5 - 3 + 2)/2 + 1 = 3**. Moreover, notice that a padding of **P = 1** is applied to the input volume, making the outer border of the input volume zero

2. The Pooling Layer (or Subsampling)

- It is common to periodically insert a Pooling layer in-between successive Conv layer in a CNN architecture. Its function is to progressively reduce the spatial size of the representation to reduce the number of parameters and computation in the network, and hence to also control overfitting.
- **Purpose:**
  - It reduces the number of parameters and the amount of computation in the network.

- It makes the feature detection more robust. By summarizing a small region, it makes the network less sensitive to the exact location of a feature (a property called translation invariance).

➢ **How Pooling works:** The Pooling layer operates independently on every depth slice of the input and resizes it spatially, using the **MAX operation**. The most common form is a pooling layer with filters of size $(\mathbf{2} \times \mathbf{2})$ applied with a stride of **2** down-samples every depth slice in the input by **2** along both width and height, discarding **75%** of the activations. Every **MAX** operation would in this case be taking a max over **4** numbers (little $\mathbf{2} \times \mathbf{2}$ region in some depth slice). The depth dimension remains unchanged.

➢ As Shown in Figure 8, the Pooling layer down-samples the volume spatially, independently in each depth slice of the input volume. **Left:** In this example, the input volume of size [**224 x 224 x 64**] is pooled with filter size **2**, stride **2** into output volume of size [**112 x 112 x 64**]. Notice that the volume depth is preserved. **Right:** The most common down-sampling operation is max, giving rise to **max pooling**, here shown with a stride of **2**. That is, each max is taken over **4** numbers (little $\mathbf{2} \times \mathbf{2}$ square).

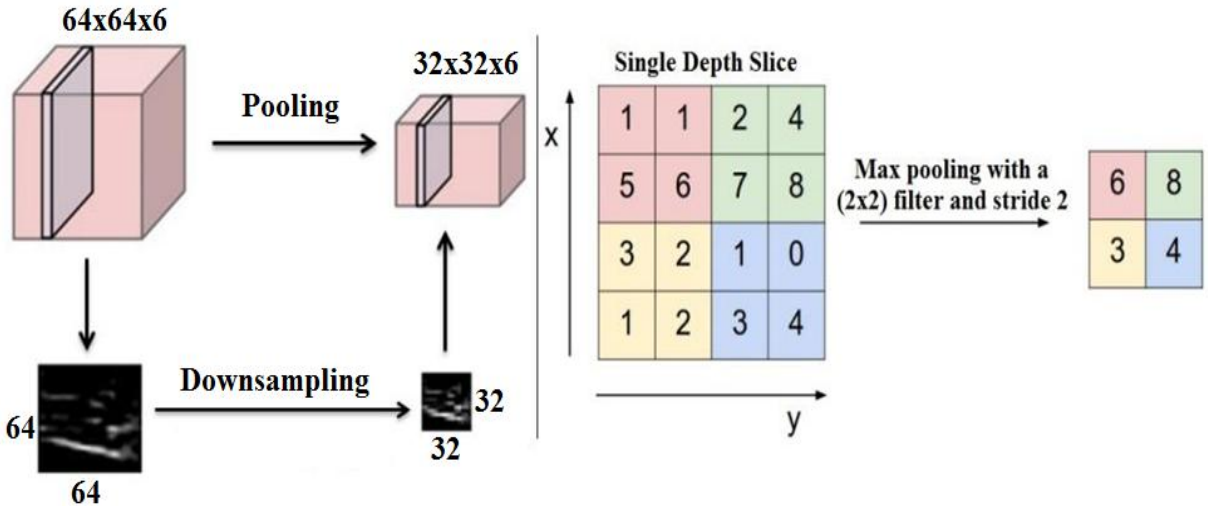


Figure 8: Pooling layer down-sample the volume spatially.

➢ More generally, the pooling layer:

- Accepts a volume of size $\mathbf{W_1 \times H_1 \times D_1}$
- Requires two hyperparameters:
  - Their spatial extent **F**,

  - The stride $\mathbf{S}$,
- Produces a volume of size $\mathbf{W_2 \times H_2 \times D_2}$ where:
  - $\mathbf{W_2 = (W_1 - F)/S + 1}$
  - $\mathbf{H_2 = (H_1 - F)/S + 1}$
  - $\mathbf{D_2 = D_1}$
- Introduces zero parameters since it computes a fixed function of the input
- For Pooling layers, it is not common to pad the input using zero-padding.

➢ It is worth noting that there are only two commonly seen variations of the max pooling layer found in practice: A pooling layer with **F = 3, S = 2** (also called **overlapping pooling**), and more commonly **F = 2, S = 2**. Pooling sizes with larger receptive fields are too destructive.

➢ **General pooling**. In addition to max pooling, the pooling units can also perform other functions, such as ***average pooling*** or even ***L2-norm pooling***. Average pooling was often used historically but has recently fallen out of favor compared to the ***max pooling*** operation, which has been shown to work better in practice.

➢ **Getting rid of pooling**. Many people dislike the pooling operation and think that we can get away without it. For example, Striving for Simplicity: The All Convolutional Net proposes to discard the pooling layer in favor of architecture that only consists of repeated CONV layers. To reduce the size of the representation they suggest using larger stride in CONV layer once in a while. Discarding pooling layers has also been found to be important in training good generative models, such as variational autoencoders (VAEs) or generative adversarial networks (GANs). It seems likely that future architectures will feature very few to no pooling layers.

3. The Fully-Connected (FC) Layer

➢ The FC layer is also known as the **Dense Layer**, is a critical component in CNNs that bridges the gap between *feature extraction* and *final prediction tasks*. While convolutional layers and pooling layers focus on identifying spatial hierarchies (e.g., edges, textures, shapes) in input data like images, the FC layer is responsible for mapping these extracted features to the output, such as class

labels in image classification, bounding boxes in object detection, or other task-specific results (See Figure 9).

- **Flattening:** The 2D feature maps from the final pooling layer are unrolled or "**flattened**" into a single long **1D vector**.
- **Classification:** This vector, which represents the most salient features of the image, is fed into one or more fully connected layers. These layers use the features to determine the probability of the image belonging to each possible class (e.g., 90% "cat", 5% "dog", 5% "bird").

➢ Neurons in a fully-connected layer have full connections to all activations in the previous layer, as seen in regular Neural Networks. Their activations can hence be computed with a *matrix multiplication* followed by ***a bias offset***.

➢ It is worth noting that the only difference between FC and CONV layers is that the neurons in the CONV layer are connected only to a *local region in the input*, and that many of the neurons in CONV volume share parameters. However, the neurons in both layers still compute **dot products**, so their functional form is identical.

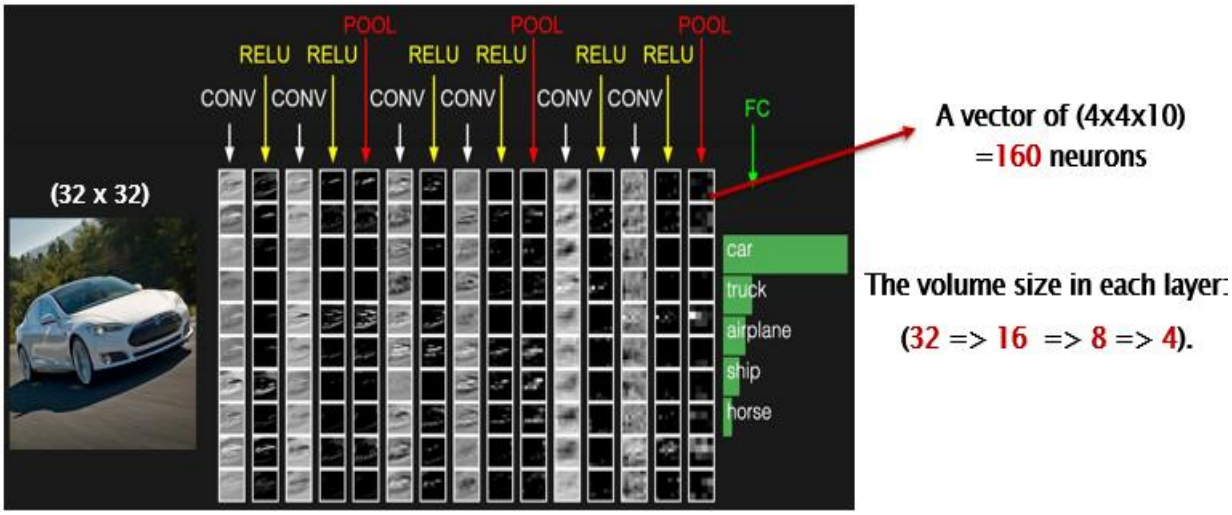


Figure 9: The activations of an example CNN architecture with different layers.

➢ **Key Characteristics of the FC layers in CNN:**

1. **Global Connectivity:** Each neuron in the FC layer receives input from all neurons of the preceding layer.

2. **Weighted Sum with Activation:** The output of each neuron is computed as a weighted sum of its inputs, followed by an activation function (e.g., ReLU, Softmax).

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$a = f(z)$$

Where:

- x_i: Input values from the previous layer.
- w_i: Weights associated with each input.
- b: Bias term.
- f: Activation function (e.g., ReLU, Softmax).

3. **Bias Terms:** Each neuron has an associated bias term that is added to the weighted sum.

- **Practical Examples of Fully-Connected Layers:** Image Classification (e.g., MNIST Dataset)

- **Input:** A (**28x28**) grayscale image (e.g., a handwritten digit).
- **Convolutional Layers:** Apply filters to detect edges, shapes, and patterns.
- **Pooling Layers:** Reduce spatial dimensions (e.g., **2x2** max pooling).
- **Flattening:** Convert the final feature map into a 1D vector (e.g., **4x4x16 → 256 elements**).
- **FC Layers:** Map the 256-dimensional vector to the output classes (e.g., **10** classes for digits **0–9**).
- **Example CNN architecture:**

```
Conv2D(32, kernel_size=3, activation='relu') →
Max-Pooling2D(2,2) → Conv2D(64, kernel_size=3,
activation='relu') → MaxPooling2D(2,2) → Flatten() →
Dense(128, activation='relu') → Dense(10,
activation='softmax')
```

The **Dense (128)** layer is a fully-connected layer with **128** neurons, and **Dense (10)** is the output layer for **10 classes**.

5. Case studies of CNNs

- There are several architectures in the field of CNNs that have a name (e.g., IrisConvNet, AlexNet, GoogLeNet, VGGNet, ResNet, etc.).
- Figure 10 shows as example of CNN named as a **IrisConvNet Model** used to address the iris recognition task.
- [ConvNetJS demo: Training on CIFAR-10]: https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html

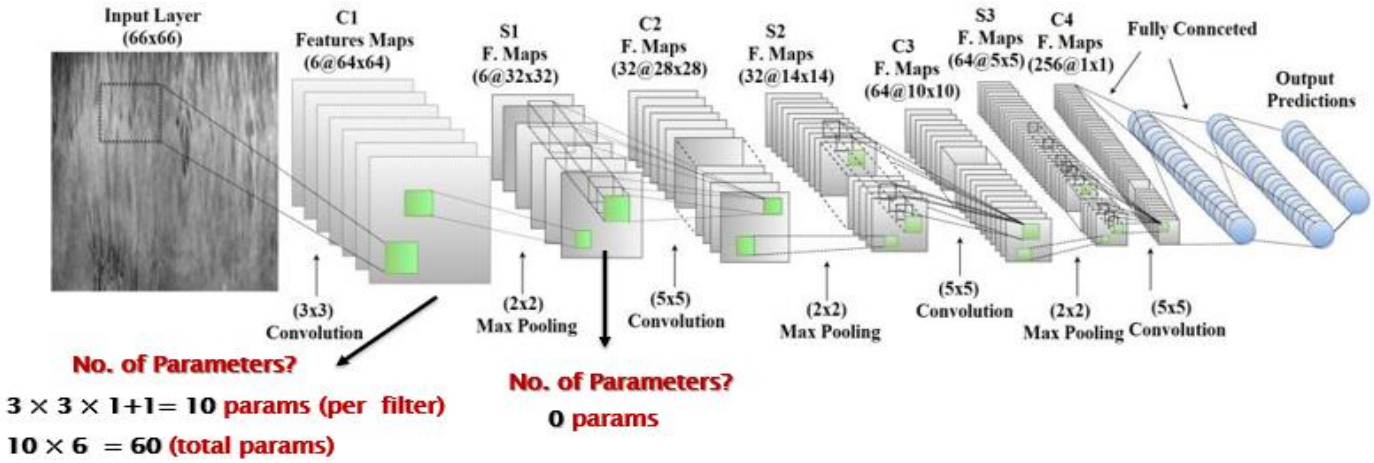


Figure 10: The main Structure of IrisConvNet Model.

6. Comprehensive Guide to Training and Deploying CNN Models

- As shown in Figure 11, a comprehensive workflow for training and deploying CNN models, structured as a circular diagram with five interconnected stages:

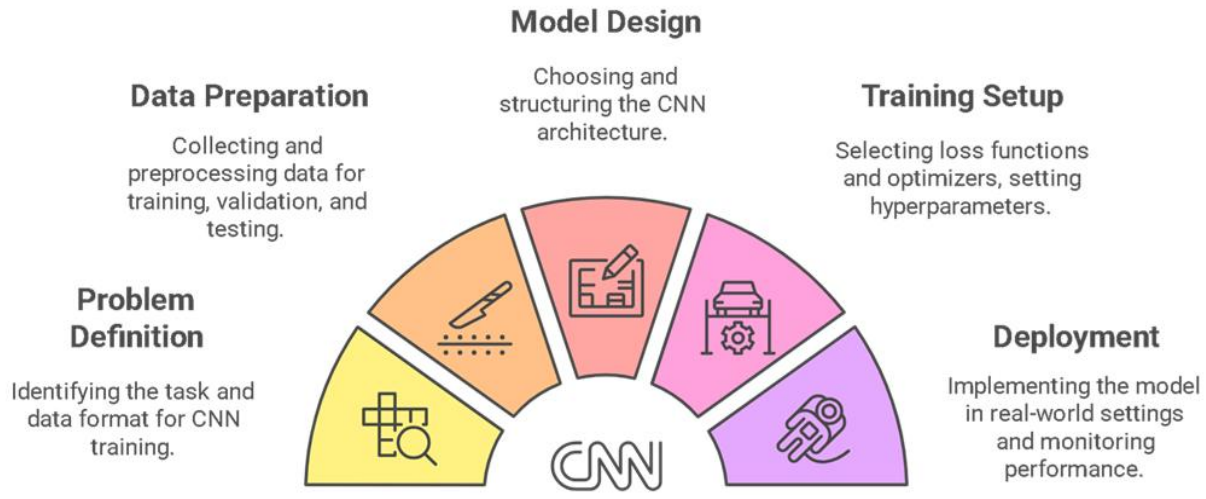


Figure 11: A comprehensive workflow for training and deploying CNN models.

1. **Problem Definition:** This is the foundational step where you identify what task the CNN will perform. This includes determining whether you're doing **image classification**, **object detection**, **segmentation**, or another computer vision task. You also decide on the data format (image dimensions, color channels, etc.) that will be used throughout the pipeline.
2. **Data Preparation:** Here you collect and preprocess your dataset. This involves gathering images, cleaning the data, applying data augmentations (rotations, flips, scaling), normalizing pixel values, and splitting the data into three sets: **training** (to learn from), **validation** (to tune hyperparameters), and **testing** (to evaluate final performance), as shown in Figure 12.

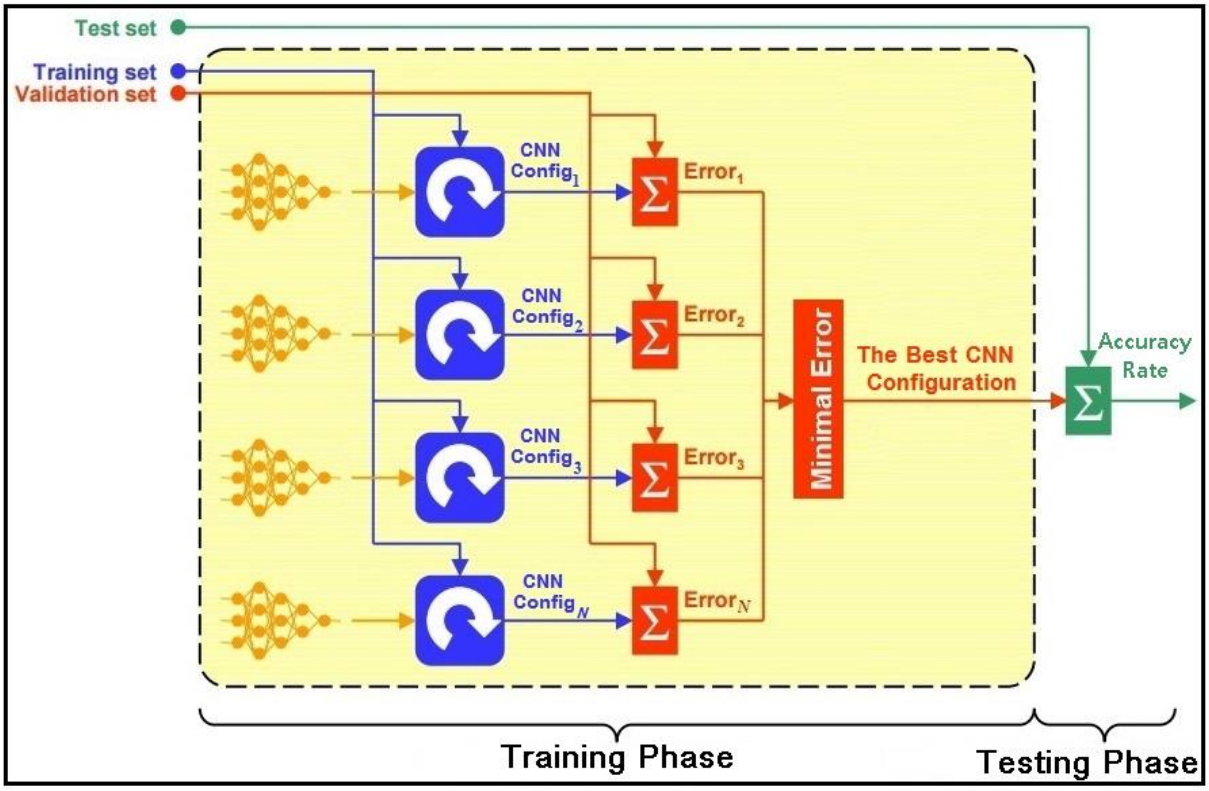


Figure 12: An overview of the training methodology to find the best trained CNN model.

3. **Model Design:** This stage focuses on choosing and structuring the CNN architecture. Decisions include the number of convolutional layers, filter sizes, pooling strategies, activation functions, and whether to use pre-trained models (transfer learning) or build from scratch.
4. **Training Setup:** You configure the training process by selecting a loss function (e.g., cross-entropy for classification), an optimizer (e.g., Adam, SGD), and setting hyperparameters like learning rate, batch size, and number of epochs.

5. Model Deployment: The final stage involves implementing the trained CNN model in real-world applications, whether on servers, edge devices, or cloud platforms, and continuously monitoring its performance to ensure reliability.

- As Shown in Figure 13, CNNs use two main training types from **scratch and transfer learning** and each fits a different data and compute situation. Training from scratch means you initialize all weights randomly and learn every feature directly from your dataset. The network learns edges first then shapes then class specific patterns as training continues. This approach needs large labeled datasets often tens or hundreds of thousands of images.
- On the other hand, Transfer learning starts from a CNN already trained on a large dataset such as ImageNet with over one million images. You reuse the learned weights because they already capture general visual features like edges textures and object parts. You either freeze most layers and train only the final layers or fine tune deeper layers with a low learning rate. This method works well with small datasets often a few hundred to a few thousand images.

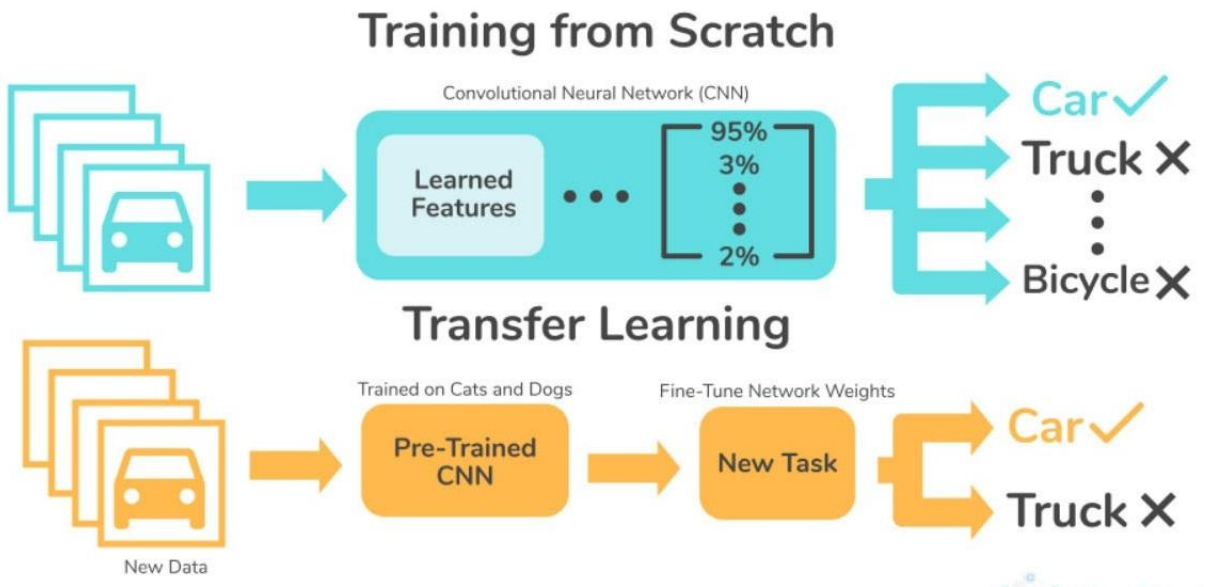


Figure 13: Training CNN models from Scratch vs. Transfer Learning.

6.1 Training a CNN from Scratch

What does "from scratch" mean?

1. Randomly initialize all network weights.
2. Train the entire network on your dataset.
3. The model learns features specific to your problem.

When to train from scratch:

1. You have a large dataset (typically 10,000+ images per class).
2. Your problem domain is very different from common datasets (ImageNet).
3. You have sufficient computational resources.
4. Example: Medical imaging with specialized modalities.

Complete Practical Example: Image Classification from Scratch

- The training pipeline consists of several interconnected components: **data loading** and **preprocessing**, **model architecture definition**, **loss function selection**, **optimizer configuration**, and **the training loop itself (e.g., number of epochs)**.
- We'll use the classic MNIST dataset with 10 classes (e.g., **0, 1, 2, …, 9**) and PyTorch, a popular deep learning framework. (See Example Code).
- Let's build a CNN to classify the CIFAR-10 dataset, which contains 60,000 32×32 color images across 10 classes (e.g., **'airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship',** and **'truck'**). (See Example Code)

6.2 Training a CNN Using Transfer Learning Strategy

What is Transfer Learning?

- Transfer learning is a machine learning technique where a model trained on one task is reused as the starting point to address a different task. As shown in Figure 14, the fundamental idea is that features learned from large datasets (like ImageNet with its 1.2 million images across 1000 classes) can be transferred to new problems, even when you have limited data.
- The intuition behind this approach comes from understanding what CNNs learn at different layers. Early convolutional layers typically learn low-level features such as edges, textures, and color gradients - these features are remarkably universal across different image domains. Middle layers capture more complex patterns like shapes and object parts. Only the final layers become highly specialized to the original task.

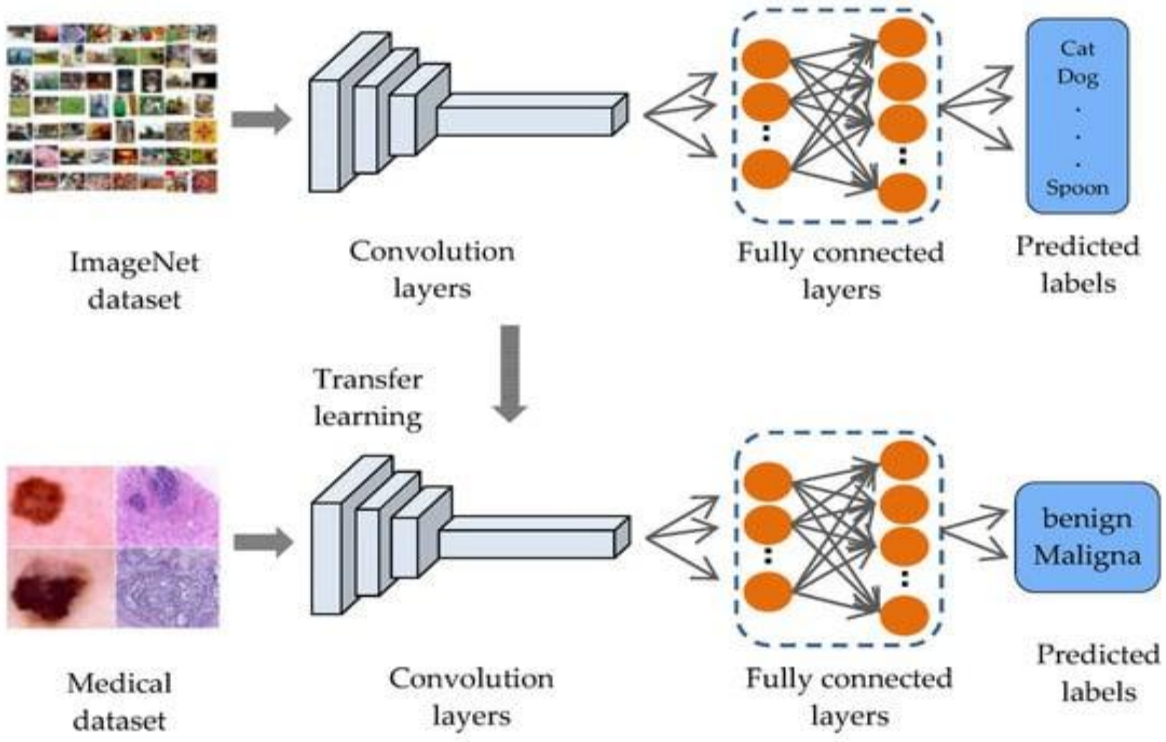


Figure 14: The transfer learning strategy in training CNNs.

The Advantages of Transfer Learning:

1. Requires less training data.
2. Faster training time.
3. Better generalization.
4. Higher accuracy on small dataset.

Two Main Transfer Learning Strategies:

- **Strategy 1: Feature Extraction** involves using the pre-trained network as a fixed feature extractor. As shown in Figure 15, you freeze all convolutional layers (preventing their weights from updating) and only train a new classifier on top. This approach is fast, requires minimal data, and works well when your dataset is small but similar to the original training data.

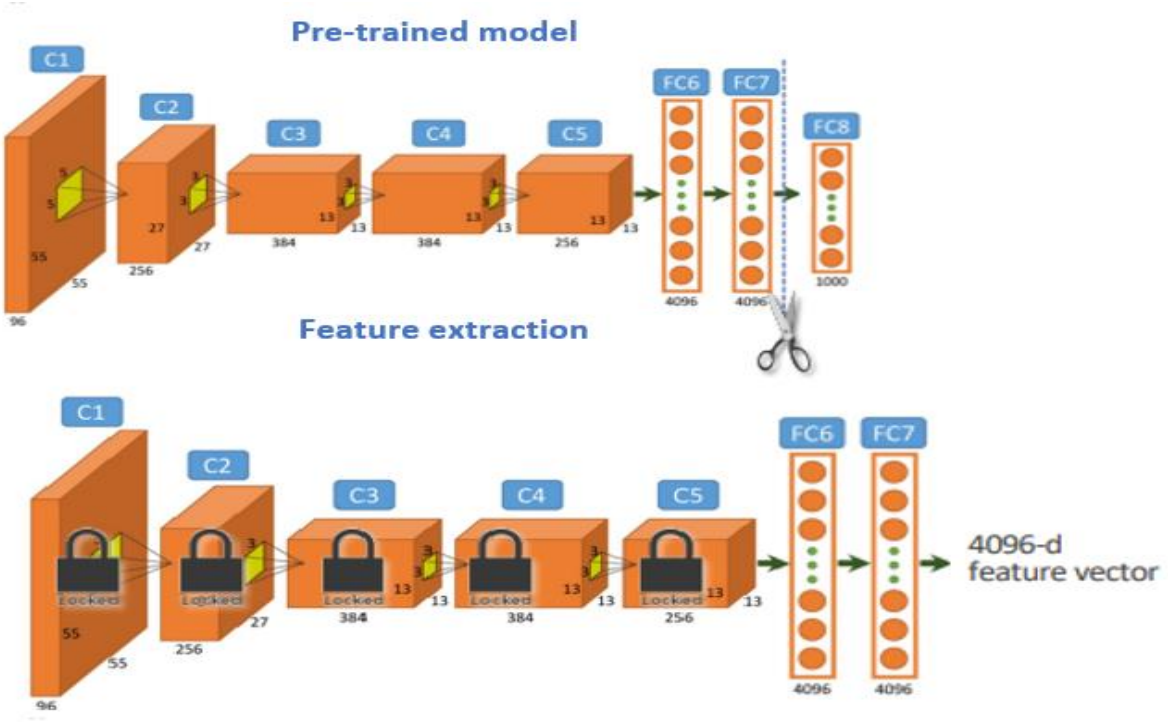


Figure 15: The transfer learning strategy used to train a CNN model as features extractor.

- **Strategy 2: Fine-Tuning** involves unfreezing some or all of the pre-trained layers and training them with a very low learning rate (See Figure 16). This allows the network to adapt its learned features to your specific domain. Fine-tuning is more powerful but requires more data and careful hyperparameter tuning to avoid catastrophic forgetting.

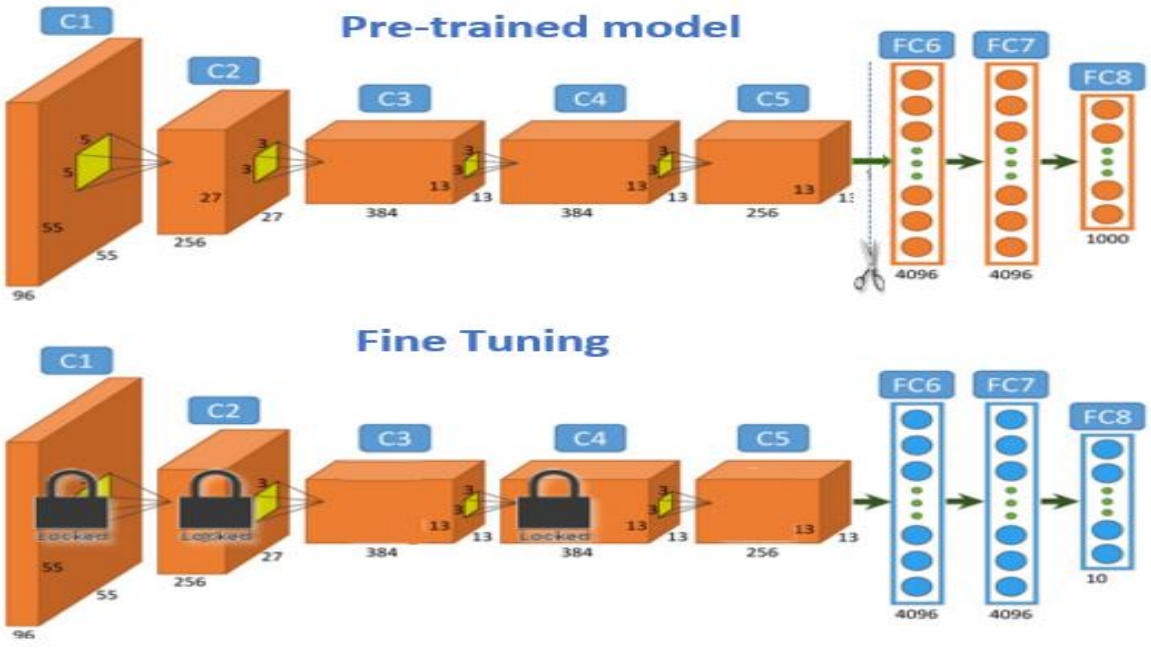


Figure 16: The transfer learning strategy used to train a CNN model as Fine-Tuning.

Here is a clear list of widely used pretrained CNN models.

- **ResNet**
  Introduced residual connections.
  Common variants: ResNet18, ResNet34, ResNet50, ResNet101.
  Strong baseline for image classification and detection.
- **VGG**
  Simple and deep architecture.
  Common variants: VGG16, VGG19.
  Used for feature extraction and transfer learning.
- **Inception**
  Uses multi scale convolution blocks.
  Common variants: Inception v1, v3.
  Efficient for high accuracy with fewer parameters.
- **MobileNet**
  Designed for mobile and edge devices.
  Common variants: MobileNet v1, v2, v3.
  Low latency and small model size.
- **DenseNet**
  Uses dense layer connections.
  Common variants: DenseNet121, DenseNet169.
  Strong feature reuse and gradient flow.
- **EfficientNet**
  Scales depth, width, and resolution together.
  Common variants: EfficientNet B0 to B7.
  High accuracy per parameter.
- **AlexNet**
  Early deep CNN model.
  Triggered modern deep learning adoption.

Complete Practical Example: Image Classification from Scratch

- **Strategy 1 - Feature Extraction:** Use a pretrained CNN (like ResNet) as a fixed feature extractor and train only the new classifier layers for a smaller custom task. For example, classifying images from CIFAR-10 (See Example Code).

🔍 Key Concepts:

1. Strategy 1 – Feature extraction: We freeze the pretrained model's weights so only the classifier (new final layer) updates (See Figure 15). This keeps the rich learned representations intact while tailoring the output to the new task.

- **Why resize to (224×224)?** Pretrained models were trained on ImageNet, which uses (**224×224**) inputs.
- **Re-normalization**: Using the same mean/std as ImageNet helps the pretrained filters respond as expected.

2. Strategy 2 – Fine-Tune: In this practical example of **transfer learning with fine-tuning**, where we not only train a new classifier head but also **unfreeze and update some (or all) of the pretrained layers** with a low learning rate. This is useful when your new dataset is somewhat similar to the original training data (e.g., ImageNet) but it has its own subtle differences (See Example Code).

- **Why two phases?**
  - **Phase 1 (Feature Extraction):** Lets the new classifier adapt to the features from the frozen backbone.
  - **Phase 2 (Fine-Tuning):** Gently adjusts deeper layers to better suit the new data.
- **Low learning rate:** Prevents large, destructive updates to the already useful pretrained weights.
- **Selective unfreezing:** Deeper layers (like layer3 and layer4 in ResNet) capture higher-level features that are more task-specific. Shallower layers (edges, textures) are often kept frozen.

6.3 Some Useful Training Strategies

❖ A number of useful training practises are used to prevent the overfitting problem during the learning process, increase the generalization ability of the neural network, and speed up convergence during training. Some of these training practises are explained below:

1. **Dropout Technique:** Dropout is a regularization technique that randomly "drops" (sets to zero) a fraction of neurons during each training iteration , as shown in Figure 17.

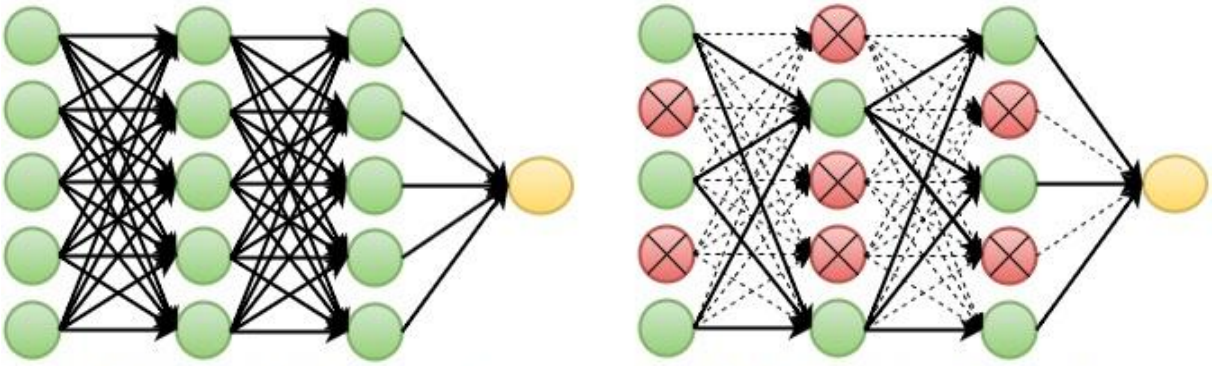


Figure 17: An illustration of applying the dropout method to a standard neural network: (a) A standard neural network with 2 hidden layers before applying dropout method. (b) An example of a reduced neural network after applying dropout method. The crossed units and the dashed connections have been dropped.

How it works:

- During training, each neuron has a probability ***p*** (e.g., **0.5**) of being temporarily disabled.
- This forces the network to not rely on any single neuron, distributing learned representations across multiple neurons.
- During inference, all neurons are active, but outputs are scaled accordingly.

Benefits:

- Prevents overfitting by reducing neuron co-dependency.
- Acts like training an ensemble of smaller networks.
- Simple to implement with minimal computational overhead.

2. Data Augmentation: Data augmentation artificially expands the training dataset by applying transformations to existing images. Figure 18 shows examples of applying different data augmentation to a colored image.

Common data augmentation methods:

- **Geometric:** rotation, vertical and horizontal flipping, cropping, scaling, translation.
- **Color:** brightness adjustment, contrast changes, saturation shifts.

Benefits:

- Increases dataset diversity without collecting new data.
- Improves generalization to unseen images.
- Reduces overfitting, especially with small datasets.

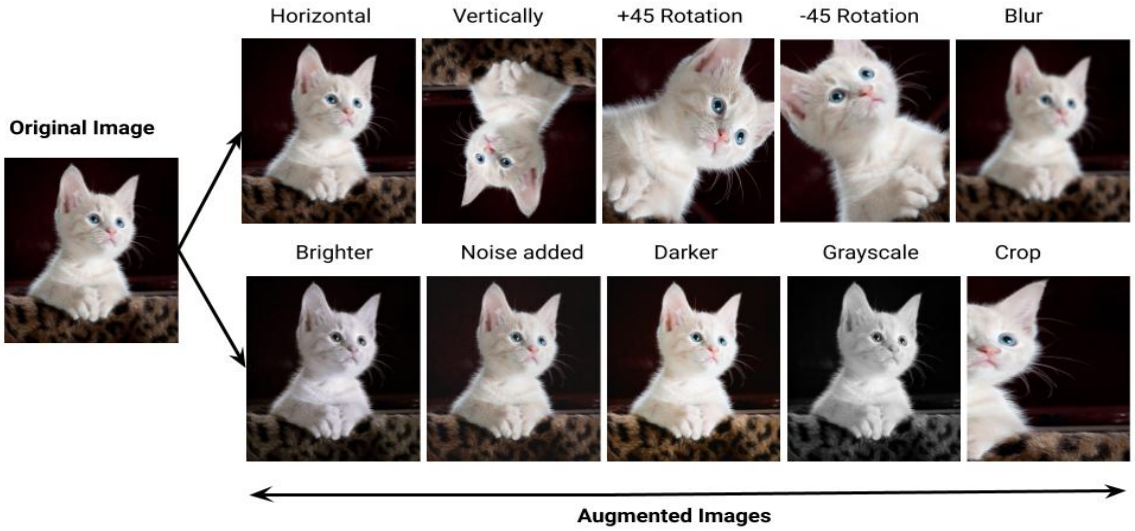


Figure 18: Examples of applying different data augmentation to a colored image.

3. Early stopping is a technique used during the training of a CNN to monitors validation loss and stops training when performance stops improving (See Figure 19).

How it works:

- Track validation loss after each epoch
- If no improvement for a set number of epochs (patience), stop training
- Restore weights from the best-performing epoch

Benefits:

- Prevents overfitting by not training longer than necessary
- Saves computational resources and **training time**

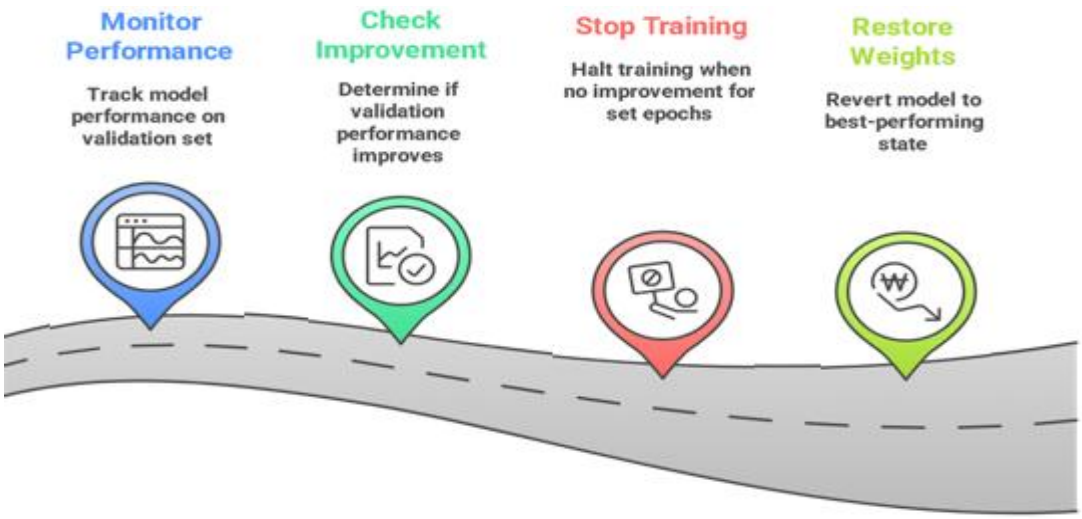


Figure 19: The main steps of applying the Early stopping procedure during the training process of a CNN model.